Podcast Q&A Bot

AI-Powered Podcast Question Answering System

Using Retrieval-Augmented Generation (RAG), FAISS, and Gemini

---

Project Report

Submitted By

Vaibhav Chauhan

Sportomic AI Lab Internship Assignment

GitHub Repository

https://github.com/Chauhanvaibhav455/podcast-qa-bot-rag

June 6, 2026

Contents

1 Abstract

Podcasts have emerged as a significant source of knowledge and information; however, locating specific topics or discussions within long-form audio content can be challenging and time-consuming. This project presents an AI-powered Podcast QA Bot designed to enable users to interact with podcast content through natural language queries and receive accurate, context-aware responses along with relevant timestamps.

The system is built using a Retrieval-Augmented Generation (RAG) architecture that combines semantic search and large language models to improve answer quality and reliability. Podcast transcripts are converted into vector embeddings using Sentence Transformers and indexed using FAISS for efficient similarity-based retrieval. When a user submits a query, the most relevant transcript segments are retrieved and provided as context to Google Gemini 2.5 Flash, which generates informative and grounded responses.

A user-friendly interface developed with Streamlit allows seamless interaction with the system. By integrating transcript extraction, semantic retrieval, vector search, and generative AI, the proposed solution enables efficient exploration of lengthy podcast content and significantly reduces the time required to locate relevant information. The system additionally provides direct YouTube navigation links that open the podcast at the most relevant timestamp associated with the generated answer.

2 Introduction

The popularity of podcasts has increased significantly over recent years. Many podcasts exceed one hour in duration and contain discussions across multiple topics. Traditional keyword-based search methods are ineffective in identifying contextually relevant information.

This project addresses this challenge by creating a semantic question-answering system capable of understanding podcast content and retrieving relevant information based on user intent rather than exact keyword matching.

3 Problem Statement

The objective of this project is to build an intelligent podcast assistant capable of:

- Understanding podcast content.
- Answering user questions using natural language.
- Retrieving relevant transcript segments.
- Returning accurate timestamps.
- Providing source evidence for answer verification.

4 Objectives

- Build a semantic search system for podcast transcripts.
- Implement Retrieval-Augmented Generation (RAG).
- Generate context-aware answers using a Large Language Model.
- Retrieve timestamps associated with relevant transcript sections.
- Create an intuitive web interface for user interaction.

5 System Architecture

Figure 5 presents the overall architecture of the proposed system.

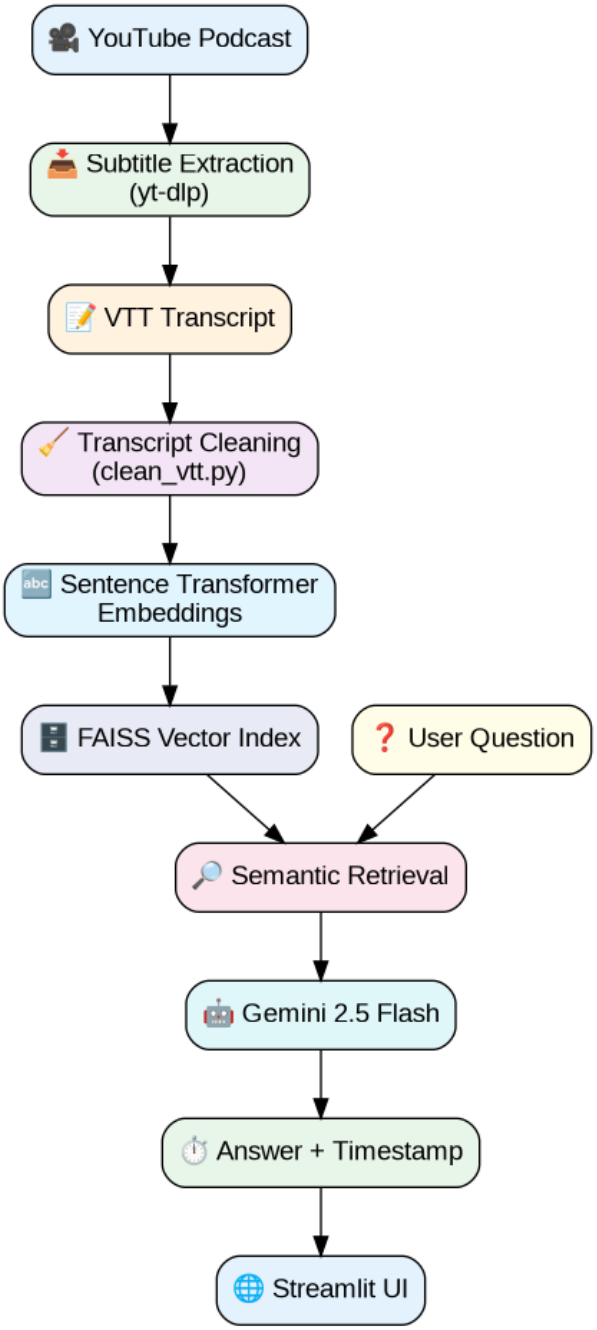

Figure 1: System Architecture

6 Technology Stack

Component	Technology Used
Programming Language	Python 3.14
Frontend	Streamlit
Embedding Model	Sentence Transformers
Vector Database	FAISS
Large Language Model	Gemini 2.5 Flash
Transcript Extraction	yt-dlp
Data Storage	Pickle Files
Version Control	Git and GitHub

Table 1: Technology Stack

7 Methodology

7.1 Transcript Extraction

The podcast transcript was extracted from YouTube using the yt-dlp utility. Auto-generated subtitles were downloaded in VTT format.

7.2 Transcript Cleaning

The raw subtitle file contained duplicate entries, timestamps, formatting tags, and metadata. A preprocessing script was developed to clean and normalize the transcript.

7.3 Embedding Generation

The cleaned transcript was divided into chunks and converted into dense vector embeddings using the Sentence Transformers library.

7.4 Vector Database Creation

Generated embeddings were stored inside a FAISS index for efficient semantic similarity search.

7.5 Question Processing

When a user enters a question, the same embedding model converts the question into a vector representation.

7.6 Semantic Retrieval

The FAISS index retrieves the most semantically similar transcript chunks.

7.7 Answer Generation

The retrieved transcript context is sent to Gemini 2.5 Flash, which generates the final answer and identifies the most relevant timestamp.

8 Implementation Workflow

The complete workflow is shown below:

1. Extract subtitles from YouTube.
2. Clean transcript data.
3. Generate embeddings.
4. Create FAISS index.
5. Accept user question.
6. Generate question embedding.
7. Perform semantic retrieval.
8. Retrieve relevant transcript chunk.
9. Generate answer using Gemini.
10. Return answer with timestamp.

9 Project Structure

```
podcast-qa-bot-rag/

 app.py
 ingest.py
 clean_vtt.py
 transcript.txt
 podcast.index
 chunks.pkl
 requirements.txt
 README.md

 assets/
    architecture.png
    home.png

 docs/
 project_report.pdf
```

10 User Interface

Figure 10 shows the final Streamlit application.

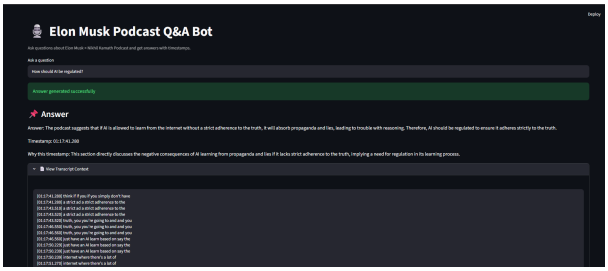

Figure 2: Podcast QA Bot Interface

11 Results

The developed system successfully retrieves relevant podcast information and generates contextual responses.

11.1 Sample Query

Question:

How should AI be regulated?

Answer:

The podcast suggests that AI systems should be aligned with truth and should avoid learning misinformation and propaganda from unreliable sources.

Timestamp Retrieved:

01:17:41

11.2 Additional Sample Questions

- What does Elon think about AI?
- What advice does Elon give entrepreneurs?
- What is the future of AI?
- Where would Elon invest?
- What does Elon say about productivity?

12 Advantages

- Fast semantic retrieval.
- Context-aware answers.
- Timestamp-based verification.
- Scalable architecture.
- User-friendly interface.
- Explainable results through transcript evidence.

13 Challenges Faced

- Noisy auto-generated subtitles.
- Duplicate transcript entries.
- Chunk size optimization.
- Retrieval accuracy tuning.
- Prompt engineering for Gemini.
- Timestamp extraction consistency.

14 Limitations

- Auto-generated subtitles may contain transcription errors.
- Some questions require larger context windows.
- Retrieval quality depends on transcript chunking.
- Timestamp accuracy depends on subtitle alignment.
- Semantic retrieval may occasionally retrieve adjacent topics.

15 Future Enhancements

- Multi-podcast support.
- Speaker identification.
- Confidence scoring.
- Hybrid retrieval.
- Conversation memory.
- Cloud deployment.
- Real-time podcast ingestion.

16 GitHub Repository

Repository Link:
https://github.com/Chauhanvaibhav455/podcast-qa-bot-rag

17 Conclusion

This project demonstrates the practical application of Retrieval-Augmented Generation (RAG) for podcast question answering. By combining semantic embeddings, FAISS vector search, and Gemini 2.5 Flash, the system enables efficient retrieval of information from long-form podcast content.

The solution successfully answers user queries, retrieves relevant timestamps, and provides supporting transcript context. The project highlights how modern AI systems can significantly improve information discovery and accessibility within multimedia content.

18 References

1. FAISS Documentation
2. Sentence Transformers Documentation
3. Google Gemini API Documentation
4. Streamlit Documentation
5. yt-dlp Documentation